Cours de C++

Guide complet pour la révision

S2 - Licence Informatique

Arthur

10 février 2026

Table des matières

1 Introduction au C++ **5**
- 1.1 Caractéristiques du langage . . . 5
- 1.2 Structure minimale d'un programme . . . 5
- 1.3 Entrées et sorties . . . 5

2 Les types de base et les tableaux **6**
- 2.1 Types de données . . . 6
- 2.2 Déclaration de tableaux . . . 6

3 Pointeurs et références **6**
- 3.1 Les pointeurs . . . 6
- 3.2 Schéma mémoire . . . 7
- 3.3 Les références . . . 7

4 Allocation dynamique **7**
- 4.1 La pile (Stack) et le tas (Heap) . . . 7
- 4.2 Allocation dans la pile . . . 7
- 4.3 Allocation dans le tas avec new/delete . . . 8
- 4.4 Exemple complet . . . 8

5 Passage de paramètres **9**
- 5.1 Passage par valeur . . . 9
- 5.2 Passage par référence . . . 9
- 5.3 Passage par référence constante . . . 10
- 5.4 Tableau comparatif . . . 10

6 Programmation orientée objet **10**
- 6.1 Concepts fondamentaux . . . 10
- 6.2 Structure d'une classe . . . 10
- 6.3 Implémentation . . . 11
- 6.4 Utilisation . . . 12

7 Constructeurs et destructeurs **13**
- 7.1 Le constructeur . . . 13
- 7.2 Types de constructeurs . . . 13
- 7.3 Utilisation . . . 14
- 7.4 Le destructeur . . . 14

8 Constructeur par recopie **15**
- 8.1 Le problème . . . 15
- 8.2 La solution : constructeur par recopie . . . 15
- 8.3 Quand est-il appelé ? . . . 16

9 Opérateur d'affectation **17**
- 9.1 Différence avec le constructeur par recopie . . . 17
- 9.2 Implémentation . . . 17
- 9.3 La règle des trois . . . 18

10 Variables const et static **18**
10.1 Variables const . . . 18
10.2 Fonctions membres const . . . 18
10.3 Variables membres static . . . 19
10.4 Exemple : compteur d'objets . . . 20

11 Valeurs par défaut et surcharge **21**
11.1 Valeurs par défaut . . . 21
11.2 Surcharge de fonctions . . . 21

12 Listes chaînées **22**
12.1 Concept . . . 22
12.2 Structure de base . . . 23
12.3 Implémentation . . . 24
12.4 Utilisation . . . 25
12.5 Schéma d'exécution . . . 26
12.6 Opérations supplémentaires . . . 26

13 Récursivité **27**
13.1 Notion de récursivité . . . 27
13.2 Récursivité croisée (ou mutuelle) . . . 27

14 Structures chaînées – Arbres **28**
14.1 Introduction . . . 28
14.2 Définition récursive . . . 28
14.3 Vocabulaire . . . 28
14.4 Caractéristiques . . . 29
14.5 Arbres ordonnés . . . 29
14.6 Arbres binaires . . . 29
14.7 Implémentation des arbres binaires . . . 29
14.8 Création d'un arbre . . . 30
14.9 Parcours des arbres . . . 31
14.10 Exemple de déroulement . . . 31
14.11 Les trois types de parcours . . . 32
14.12 Répartition des fonctions entre Arbre et Nœud . . . 32
14.13 Arbres binaires ordonnés . . . 33
14.14 Quelques arbres particuliers . . . 33

15 Bonnes pratiques et pièges à éviter **34**
15.1 Checklist des bonnes pratiques . . . 34
15.2 Pièges fréquents . . . 35
15.3 Conseils de débogage . . . 36

16 Exercices d'entraînement **36**
16.1 Exercice 1 : Classe Rectangle . . . 36
16.2 Exercice 2 : Classe Vecteur dynamique . . . 37
16.3 Exercice 3 : Pile (Stack) avec liste chaînée . . . 39

17 Aide-mémoire **41**
17.1 Tableau récapitulatif des opérateurs 41
17.2 Commandes de compilation 41
17.3 Modèle de fichiers .h et .cpp 41

18 Conclusion **42**

1 Introduction au C++

1.1 Caractéristiques du langage

Le C++ est un langage **compilé**, c'est-à-dire qu'avant d'être exécuté, le code source doit être traduit en langage machine par un compilateur.

Exemple

Processus de compilation et exécution :

1. Écriture du code dans `programme.cpp`
2. Compilation : `g++ programme.cpp -o programme`
3. Exécution : `./programme`

1.2 Structure minimale d'un programme

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!" << endl;
    return 0;
}
```

Listing 1 – Programme minimal en C++

- `#include <iostream>` : inclut la bibliothèque d'entrées/sorties
- `using namespace std;` : permet d'utiliser `cout` et `cin` sans préfixe
- `int main()` : fonction principale, point d'entrée du programme

1.3 Entrées et sorties

En C++, on utilise `cout` et `cin` à la place de `printf` et `scanf`.

```
int age;
string nom;

// Affichage
cout << "Entrez votre nom : ";
cin >> nom;

cout << "Entrez votre age : ";
cin >> age;

cout << "Bonjour " << nom << ", vous avez " << age << " ans."
    << endl;
```

Listing 2 – Utilisation de cout et cin

Règle

Opérateurs de flux :
- « : pour envoyer vers la sortie (`cout`)
- » : pour récupérer depuis l'entrée (`cin`)
- `endl` : retour à la ligne

2 Les types de base et les tableaux

2.1 Types de données

Table 1 – Types de données en C++

Type	Description	Exemple
`int`	Entier	`int x = 42;`
`double`	Réel double précision	`double pi = 3.14;`
`float`	Réel simple précision	`float f = 2.5f;`
`char`	Caractère	`char c = 'A';`
`bool`	Booléen	`bool ok = true;`
`string`	Chaîne de caractères	`string s = "hello";`

2.2 Déclaration de tableaux

```
// Tableau a une dimension
int tab[10];        // Tableau de 10 entiers
tab[0] = 5;         // Premier element (indice 0)
tab[9] = 12;        // Dernier element (indice 9)

// Tableau a deux dimensions
double matrice[3][4]; // 3 lignes, 4 colonnes
matrice[0][0] = 1.5;

// Initialisation lors de la declaration
int nombres[] = {10, 20, 30, 40, 50};
```

Listing 3 – Tableaux statiques

Important

Les tableaux statiques ont une taille fixe définie à la compilation. Cette taille ne peut pas être modifiée pendant l'exécution du programme.

3 Pointeurs et références

3.1 Les pointeurs

Un pointeur est une variable qui contient l'**adresse mémoire** d'une autre variable.

```
int x = 42;
int* p;          // p est un pointeur sur int

p = &x;          // p contient l'adresse de x

cout << x;       // Affiche : 42
cout << &x;      // Affiche : 0x7fff5a3b524 (adresse de x)
cout << p;       // Affiche : 0x7fff5a3b524 (meme adresse)
cout << *p;      // Affiche : 42 (valeur pointee)
```

Listing 4 – Déclaration et utilisation de pointeurs

Règle

Les deux opérateurs clés :
- `&` : opérateur d'adresse, donne l'adresse d'une variable
- `*` : opérateur de déréférencement, accède à la valeur pointée

3.2 Schéma mémoire

Variable	Adresse	Valeur
x	0x1000	42
p	0x2000	0x1000

Le pointeur `p` (stocké à l'adresse `0x2000`) contient l'adresse de `x` (`0x1000`).

3.3 Les références

Une référence est un **alias** pour une variable existante.

```
int x = 10;
int& ref = x;    // ref est une reference sur x

ref = 20;        // Modifie x
cout << x;       // Affiche : 20
```

Listing 5 – Utilisation des références

Astuce

Une référence doit être initialisée lors de sa déclaration et ne peut plus changer de cible par la suite.

4 Allocation dynamique

4.1 La pile (Stack) et le tas (Heap)

4.2 Allocation dans la pile

Table 2 – Comparaison Pile vs Tas

	Pile (Stack)	**Tas (Heap)**
Taille	Limitée (∼8 Mo)	Quasi-illimitée
Libération	Automatique	Manuelle (`delete`)
Utilisation	Variables locales	Allocation dynamique
Rapidité	Rapide	Plus lent

```
void fonction() {
    int x = 5;          // Cree dans la pile
    double tab[10];      // Cree dans la pile
    // ...
} // x et tab sont automatiquement detruits ici
```

Listing 6 – Variables locales dans la pile

4.3 Allocation dans le tas avec new/delete

```
// Allocation d'un seul element
int* p = new int;
*p = 42;
delete p;            // Libere la memoire

// Allocation d'un tableau
int n = 100;
int* tableau = new int[n];
tableau[0] = 10;
tableau[99] = 20;
delete[] tableau;    // ATTENTION aux crochets !
```

Listing 7 – Allocation dynamique avec new

Important

Règle d'or :
- Chaque `new` doit avoir son `delete`
- Chaque `new[]` doit avoir son `delete[]`
- Sinon : **fuite mémoire !**

4.4 Exemple complet

```
#include <iostream>
using namespace std;

int main() {
    int taille;
    cout << "Entrez la taille du tableau : ";
    cin >> taille;
```

```

    // Allocation dynamique
    int* tab = new int[taille];

    // Remplissage
    for (int i = 0; i < taille; i++) {
        tab[i] = i * 10;
    }

    // Affichage
    for (int i = 0; i < taille; i++) {
        cout << tab[i] << " ";
    }
    cout << endl;

    // Liberation
    delete[] tab;

    return 0;
}
```

Listing 8 – Gestion dynamique d'un tableau

5 Passage de paramètres

5.1 Passage par valeur

Le paramètre reçoit une **copie** de la variable.

```
void modifier(int x) {
    x = 10;           // Modifie la COPIE
}

int main() {
    int a = 5;
    modifier(a);
    cout << a;        // Affiche : 5 (non modifie)
}
```

Listing 9 – Passage par valeur

5.2 Passage par référence

Le paramètre est un **alias** de la variable originale.

```
void modifier(int& x) {   // Le & est crucial
    x = 10;           // Modifie l'ORIGINAL
}

int main() {
    int a = 5;
```

```
    modifier(a);
    cout << a;        // Affiche : 10 (modifie !)
}
```

Listing 10 – Passage par référence

5.3 Passage par référence constante

Permet d'éviter la copie tout en protégeant la variable.

```
void afficher(const int& x) {
    cout << x;
    // x = 10;       // ERREUR de compilation
}
```

Listing 11 – Passage par référence constante

Règle

Quand utiliser quoi ?
- **Par valeur** : types simples (`int`, `char`, `bool`, `double`)
- **Par référence constante** (`const &`) : objets en lecture seule
- **Par référence** (`&`) : objets à modifier

5.4 Tableau comparatif

Table 3 – Modes de passage de paramètres

Mode	Copie ?	Modifiable ?	Syntaxe
Par valeur	Oui	Non (copie)	`void f(int x)`
Par référence	Non	Oui	`void f(int& x)`
Par réf. const.	Non	Non	`void f(const int& x)`

6 Programmation orientée objet

6.1 Concepts fondamentaux

La **Programmation Orientée Objet** (POO) permet de :
- Regrouper données et fonctions dans une même structure (classe)
- Protéger les données (encapsulation)
- Créer des types personnalisés
- Améliorer la maintenabilité et la réutilisabilité du code

6.2 Structure d'une classe

```cpp
#ifndef POINT_H
#define POINT_H

class Point {
private:      // Donnees membres privees
    double x;
    double y;

public:       // Fonctions membres publiques
    // Constructeur
    Point(double abs, double ord);

    // Methodes
    void afficher() const;
    void deplacer(double dx, double dy);

    // Accesseurs (getters)
    double getX() const;
    double getY() const;

    // Mutateurs (setters)
    void setX(double valeur);
    void setY(double valeur);
};

#endif
```

Listing 12 – Déclaration d'une classe - fichier Point.h

6.3 Implémentation

```cpp
#include "Point.h"
#include <iostream>
using namespace std;

// Constructeur
Point::Point(double abs, double ord) {
    x = abs;
    y = ord;
}

void Point::afficher() const {
    cout << "(" << x << ", " << y << ")" << endl;
}

void Point::deplacer(double dx, double dy) {
    x += dx;
    y += dy;
}

```

```
double Point::getX() const {
    return x;
}

double Point::getY() const {
    return y;
}

void Point::setX(double valeur) {
    x = valeur;
}

void Point::setY(double valeur) {
    y = valeur;
}
```

Listing 13 – Implémentation de la classe - fichier Point.cpp

6.4 Utilisation

```
#include "Point.h"
#include <iostream>
using namespace std;

int main() {
    // Creation d'objets
    Point p1(3.0, 5.0);
    Point p2(1.0, 2.0);

    // Utilisation des methodes
    p1.afficher();          // (3, 5)
    p1.deplacer(2.0, 1.0);
    p1.afficher();          // (5, 6)

    // Utilisation des accesseurs
    cout << "x = " << p1.getX() << endl;

    // Utilisation des mutateurs
    p1.setX(10.0);
    p1.afficher();          // (10, 6)

    // INTERDIT : les donnees sont privees
    // p1.x = 20;           // ERREUR de compilation

    return 0;
}
```

Listing 14 – Programme principal - fichier main.cpp

Règle

Principe d'encapsulation :
- Les données membres sont `private`
- Les méthodes d'interface sont `public`
- On accède aux données via des accesseurs (getters) et mutateurs (setters)

7 Constructeurs et destructeurs

7.1 Le constructeur

Le constructeur est une fonction appelée **automatiquement** lors de la création d'un objet.

Règle

Caractéristiques :
- Porte le même nom que la classe
- N'a pas de type de retour
- Sert à initialiser l'objet
- Peut effectuer des allocations dynamiques

7.2 Types de constructeurs

```cpp
class Tableau {
private:
    int* tab;
    int taille;

public:
    // 1) Constructeur par defaut
    Tableau() {
        taille = 10;
        tab = new int[10];
    }

    // 2) Constructeur avec parametres
    Tableau(int n) {
        taille = n;
        tab = new int[n];
    }

    // 3) Constructeur avec valeur par defaut
    Tableau(int n = 10) {
        taille = n;
        tab = new int[n];
    }
};
```

Listing 15 – Différents types de constructeurs

7.3 Utilisation

```
int main() {
    Tableau t1;          // Appelle le constructeur par defaut
    Tableau t2(20);      // Appelle le constructeur avec n=20
    Tableau t3(50);      // Appelle le constructeur avec n=50

    // ATTENTION : ceci ne cree PAS un objet !
    // Tableau t4();     // Declare une fonction !
}
```

Listing 16 – Appel des constructeurs

7.4 Le destructeur

Le destructeur est une fonction appelée **automatiquement** lors de la destruction d'un objet.

Règle

Caractéristiques :
- Porte le nom de la classe précédé de ~
- N'a pas de paramètres
- N'a pas de type de retour
- Sert à libérer les ressources (mémoire, fichiers, etc.)

```
class Tableau {
private:
    int* tab;
    int taille;

public:
    Tableau(int n) {
        taille = n;
        tab = new int[n];
        cout << "Construction d'un tableau de " << n << endl;
    }

    ~Tableau() {          // Destructeur
        delete[] tab;
        cout << "Destruction du tableau" << endl;
    }
};

void fonction() {
    Tableau t(100);
    // ...
} // Le destructeur est appele ICI automatiquement
```

Listing 17 – Destructeur

Important

Si votre classe contient des pointeurs alloués avec `new`, vous **DEVEZ** écrire un destructeur pour faire les `delete` correspondants, sinon : **fuite mémoire !**

8 Constructeur par recopie

8.1 Le problème

Sans constructeur par recopie défini, le compilateur fait une **copie superficielle** des pointeurs.

```
class Tableau {
    int* tab;
    int taille;
public:
    Tableau(int n) {
        taille = n;
        tab = new int[n];
    }

    ~Tableau() {
        delete[] tab;
    }
};

int main() {
    Tableau t1(5);
    Tableau t2 = t1;      // Copie superficielle !

    // t1 et t2 pointent vers le MEME tableau
} // CRASH : double delete !
```

Listing 18 – Problème de copie superficielle

Schéma du problème :

Avant copie :
`t1.tab → [zone mémoire A]`

Après copie superficielle :
```
t1.tab → [zone mémoire A]
                ↑
t2.tab
```

À la destruction :
`~t1() fait delete[] sur zone A`
`~t2() fait delete[] sur zone A → ERREUR !`

8.2 La solution : constructeur par recopie

```
class Tableau {
```

```
private:
    int* tab;
    int taille;

public:
    // Constructeur normal
    Tableau(int n) {
        taille = n;
        tab = new int[n];
    }

    // CONSTRUCTEUR PAR RECOPIE
    Tableau(const Tableau& autre) {
        taille = autre.taille;

        // Nouvelle allocation (copie profonde)
        tab = new int[taille];

        // Copie des valeurs
        for (int i = 0; i < taille; i++) {
            tab[i] = autre.tab[i];
        }
    }

    // Destructeur
    ~Tableau() {
        delete[] tab;
    }
};
```

Listing 19 – Constructeur par recopie (copie profonde)

Règle

Prototype du constructeur par recopie :
`NomClasse(const NomClasse& autre)`

8.3 Quand est-il appelé ?

```
Tableau t1(10);

// 1) Initialisation par copie
Tableau t2 = t1;        // Appelle le constructeur par recopie
Tableau t3(t1);         // Meme chose

// 2) Passage par valeur
void fonction(Tableau t) {  // Copie !
    // ...
}
fonction(t1);           // Appelle le constructeur par recopie

```

```
// 3) Retour par valeur
Tableau creerTableau() {
    Tableau temp(5);
    return temp;        // Copie !
}
```

Listing 20 – Situations d'appel du constructeur par recopie

Astuce

Pour éviter les copies inutiles, on préfère souvent passer les objets par référence constante : `const Tableau&`

9 Opérateur d'affectation

9.1 Différence avec le constructeur par recopie

```
Tableau t1(10);

// Constructeur par recopie (creation)
Tableau t2 = t1;

// Operateur d'affectation (objets existants)
Tableau t3(5);
t3 = t1;                // Affectation
```

Listing 21 – Constructeur par recopie vs opérateur d'affectation

9.2 Implémentation

```
class Tableau {
private:
    int* tab;
    int taille;

public:
    // Operateur d'affectation
    Tableau& operator=(const Tableau& autre) {
        // 1) Verifier l'auto-affectation
        if (this == &autre) {
            return *this;
        }

        // 2) Liberer l'ancienne memoire
        delete[] tab;

        // 3) Copier les nouvelles donnees
        taille = autre.taille;
        tab = new int[taille];
        for (int i = 0; i < taille; i++) {
```

```
21                tab[i] = autre.tab[i];
22            }
23
24            // 4) Retourner *this
25            return *this;
26        }
27 };
```

Listing 22 – Opérateur d'affectation

Règle

Les 4 étapes de l'opérateur d'affectation :

1. Vérifier l'auto-affectation (`t = t`)
2. Libérer les anciennes ressources
3. Copier les nouvelles données
4. Retourner `*this`

9.3 La règle des trois

Important

Règle des trois :
Si votre classe nécessite l'un des trois, elle nécessite probablement les trois :

1. **Destructeur**
2. **Constructeur par recopie**
3. **Opérateur d'affectation**

C'est le cas pour toute classe qui gère des ressources dynamiques (pointeurs, fichiers, etc.).

10 Variables const et static

10.1 Variables const

Une variable `const` ne peut pas être modifiée après son initialisation.

```
1 const int MAX = 100;
2 MAX = 50;                // ERREUR de compilation
3
4 const double PI = 3.14159;
5 PI = 3.14;               // ERREUR de compilation
```

Listing 23 – Utilisation de const

10.2 Fonctions membres const

Une fonction membre `const` promet de ne pas modifier l'objet.

```
class Point {
private:
    double x, y;

public:
    // Methode const : ne modifie pas l'objet
    void afficher() const {
        cout << "(" << x << ", " << y << ")" << endl;
        // x = 10;       // ERREUR de compilation !
    }

    // Accesseur const
    double getX() const {
        return x;
    }

    // Mutateur non-const
    void setX(double valeur) {
        x = valeur;
    }
};
```

Listing 24 – Méthodes const

Règle

Quand utiliser `const` ?
- Pour les accesseurs (getters)
- Pour toute méthode qui ne modifie pas l'objet
- Permet d'appeler ces méthodes sur des objets `const`

10.3 Variables membres static

Une variable membre `static` est partagée par **tous** les objets de la classe.

```
class Cercle {
private:
    static double pi;      // Partagee par TOUS les cercles
    double rayon;           // Propre a chaque cercle

public:
    Cercle(double r) : rayon(r) {}

    double aire() const {
        return pi * rayon * rayon;
    }

    static void setPi(double valeur) {
        pi = valeur;
    }
};
```

```

// Initialisation (OBLIGATOIRE hors de la classe)
double Cercle::pi = 3.14159;

int main() {
    Cercle c1(5.0);
    Cercle c2(10.0);

    // c1 et c2 partagent la MEME variable pi
    Cercle::setPi(3.14);  // Modifie pour tous les cercles
}
```

Listing 25 – Variable membre static

Astuce

Les variables `static` sont utiles pour :
- Compter le nombre d'instances créées
- Partager des constantes entre tous les objets
- Implémenter le pattern Singleton

10.4 Exemple : compteur d'objets

```
class Compteur {
private:
    static int nombreObjets;  // Compte le nombre d'objets

public:
    Compteur() {
        nombreObjets++;
        cout << "Creation : " << nombreObjets << " objets" << endl;
    }

    ~Compteur() {
        nombreObjets--;
        cout << "Destruction : " << nombreObjets << " objets" << endl;
    }

    static int getNombre() {
        return nombreObjets;
    }
};

// Initialisation
int Compteur::nombreObjets = 0;

int main() {
    Compteur c1;
    {
```

```
27          Compteur c2, c3;
28          cout << "Nombre actuel : " << Compteur::getNombre() <<
    endl;
29      } // c2 et c3 sont detruits ici
30
31      cout << "Nombre final : " << Compteur::getNombre() << endl;
32  }
```

Listing 26 – Compteur avec static

11 Valeurs par défaut et surcharge

11.1 Valeurs par défaut

On peut donner des valeurs par défaut aux paramètres.

```
1  class Point {
2  public:
3      // Constructeur avec valeurs par defaut
4      Point(double x = 0.0, double y = 0.0) {
5          this->x = x;
6          this->y = y;
7      }
8  };
9
10 int main() {
11     Point p1;                // x=0, y=0
12     Point p2(5.0);           // x=5, y=0
13     Point p3(5.0, 3.0);    // x=5, y=3
14 }
```

Listing 27 – Paramètres avec valeurs par défaut

Règle

Règles pour les valeurs par défaut :
- Les paramètres avec valeur par défaut doivent être à droite
- Déclarés dans le `.h`, pas dans le `.cpp`

```
// CORRECT
void f(int a, int b = 5, int c = 10);

// INCORRECT
void f(int a = 1, int b);
```

11.2 Surcharge de fonctions

Plusieurs fonctions peuvent avoir le même nom si elles ont des paramètres différents.

```cpp
class Point {
public:
    // Version avec deux doubles
    void initialiser(double x, double y) {
        this->x = x;
        this->y = y;
    }

    // Version avec un autre Point
    void initialiser(const Point& p) {
        this->x = p.x;
        this->y = p.y;
    }

    // Version avec un pointeur sur Point
    void initialiser(const Point* p) {
        this->x = p->x;
        this->y = p->y;
    }
};

int main() {
    Point p1, p2, p3;

    p1.initialiser(5.0, 3.0);   // Appelle la 1ere version
    p2.initialiser(p1);          // Appelle la 2eme version
    p3.initialiser(&p1);         // Appelle la 3eme version
}
```

Listing 28 – Surcharge de fonctions

Astuce

Le compilateur choisit automatiquement la bonne version en fonction des arguments fournis. C'est ce qu'on appelle la **résolution de surcharge**.

12 Listes chaînées

12.1 Concept

Une liste chaînée est une structure de données où chaque élément pointe vers le suivant.

Tete → [12|→] → [33|→] → [25|/]

Règle

Avantages :
- Taille dynamique (peut grandir/rétrécir)
- Insertion/suppression en début de liste en O(1)

Inconvénients :
- Accès séquentiel uniquement (pas d'accès direct comme `tab[i]`)
- Consommation mémoire supplémentaire (pointeurs)

12.2 Structure de base

```cpp
#ifndef LISTE_H
#define LISTE_H

class Maillon {
private:
    int Info;
    Maillon* suivant;

    friend class Liste;  // Liste peut acceder aux membres prives

public:
    Maillon(int valeur = 0, Maillon* suiv = nullptr) {
        Info = valeur;
        suivant = suiv;
    }
};

class Liste {
private:
    Maillon* tete;

public:
    // Constructeur
    Liste();

    // Destructeur
    ~Liste();

    // Methodes
    void ajouterDebut(int valeur);
    void ajouterFin(int valeur);
    void afficher() const;
    bool estVide() const;
    int taille() const;
};

#endif
```

Listing 29 – Classe Maillon et Liste - fichier Liste.h

12.3 Implémentation

```cpp
#include "Liste.h"
#include <iostream>
using namespace std;

// Constructeur
Liste::Liste() {
    tete = nullptr;      // Liste vide
}

// Destructeur
Liste::~Liste() {
    while (tete != nullptr) {
        Maillon* temp = tete;
        tete = tete->suivant;
        delete temp;
    }
}

// Verifier si la liste est vide
bool Liste::estVide() const {
    return tete == nullptr;
}

// Ajouter en debut de liste
void Liste::ajouterDebut(int valeur) {
    tete = new Maillon(valeur, tete);
}

// Ajouter en fin de liste
void Liste::ajouterFin(int valeur) {
    // Cas particulier : liste vide
    if (tete == nullptr) {
        tete = new Maillon(valeur);
        return;
    }

    // Parcourir jusqu'a la fin
    Maillon* p = tete;
    while (p->suivant != nullptr) {
        p = p->suivant;
    }

    // Ajouter a la fin
    p->suivant = new Maillon(valeur);
}

// Afficher la liste
void Liste::afficher() const {
    Maillon* p = tete;
```

```

    cout << "Liste : ";
    while (p != nullptr) {
        cout << p->Info << " ";
        p = p->suivant;
    }
    cout << endl;
}

// Calculer la taille
int Liste::taille() const {
    int compteur = 0;
    Maillon* p = tete;

    while (p != nullptr) {
        compteur++;
        p = p->suivant;
    }

    return compteur;
}
```

Listing 30 – Implémentation de Liste - fichier Liste.cpp

12.4 Utilisation

```
#include "Liste.h"
#include <iostream>
using namespace std;

int main() {
    Liste L;

    // Ajouts
    L.ajouterDebut(25);
    L.ajouterDebut(33);
    L.ajouterDebut(12);

    L.afficher();        // Liste : 12 33 25

    L.ajouterFin(51);
    L.afficher();        // Liste : 12 33 25 51

    cout << "Taille : " << L.taille() << endl;

    return 0;
} // Le destructeur libere automatiquement toute la liste
```

Listing 31 – Programme principal

12.5 Schéma d'exécution

Après les ajouts en début :
L.tete → [12|→] → [33|→] → [25|/]

Après l'ajout en fin :
L.tete → [12|→] → [33|→] → [25|→] → [51|/]

12.6 Opérations supplémentaires

```
// Rechercher une valeur
bool Liste::contient(int valeur) const {
    Maillon* p = tete;
    while (p != nullptr) {
        if (p->Info == valeur)
            return true;
        p = p->suivant;
    }
    return false;
}

// Supprimer le premier element
void Liste::supprimerDebut() {
    if (tete == nullptr)
        return;

    Maillon* temp = tete;
    tete = tete->suivant;
    delete temp;
}

// Supprimer toutes les occurrences d'une valeur
void Liste::supprimer(int valeur) {
    // Supprimer en tete
    while (tete != nullptr && tete->Info == valeur) {
        Maillon* temp = tete;
        tete = tete->suivant;
        delete temp;
    }

    // Supprimer dans le reste
    if (tete == nullptr)
        return;

    Maillon* p = tete;
    while (p->suivant != nullptr) {
        if (p->suivant->Info == valeur) {
            Maillon* temp = p->suivant;
            p->suivant = temp->suivant;
            delete temp;
        } else {
```

```
            p = p->suivant;
        }
    }
}
```

Listing 32 – Autres opérations utiles

13 Récursivité

13.1 Notion de récursivité

Une fonction est dite **récursive** lorsqu'elle s'appelle elle-même. On dit qu'elle « se répète toute seule indéfiniment » (tant qu'une condition d'arrêt n'est pas atteinte).

Exemple

La factorielle ($n!$) est un exemple classique de récursivité :

$$n! = \begin{cases} 1 & \text{si } n \leq 1 \\ n \times (n-1)! & \text{sinon} \end{cases}$$

Soit : $n! = 1 \times 2 \times 3 \times 4 \times \ldots \times (n-1) \times n$

```
int factorielle(int n) {
    if (n <= 1)
        return 1;
    else
        return n * factorielle(n - 1);
}
```

Listing 33 – Fonction factorielle récursive

Règle

Toute fonction récursive doit posséder :
- Un **cas de base** (condition d'arrêt) qui stoppe la récursion
- Un **appel récursif** qui se rapproche du cas de base à chaque itération

Sans condition d'arrêt, la fonction s'appelle indéfiniment → débordement de pile !

13.2 Récursivité croisée (ou mutuelle)

On parle de **récursivité croisée** (ou mutuelle) lorsque deux fonctions s'appellent mutuellement. Un exemple classique est le test de parité/imparité avec des fonctions booléennes :

```
bool estPair(int n);    // Declaration anticipee
bool estImpair(int n);

bool estPair(int n) {
    if (n == 0)
```

```
        return true;
    return estImpair(n - 1);
}

bool estImpair(int n) {
    if (n == 0)
        return false;
    return estPair(n - 1);
}
```

Listing 34 – Récursivité croisée : parité et imparité

Astuce

La récursivité croisée nécessite une **déclaration anticipée** d'au moins une des deux fonctions, car chacune doit connaître la signature de l'autre pour pouvoir l'appeler.

14 Structures chaînées – Arbres

14.1 Introduction

On peut voir un arbre comme une **généralisation de listes chaînées** qui porteraient comme informations des listes. De manière plus intuitive, un arbre est la modélisation d'une **organisation hiérarchique** de données.

Exemple

Exemples d'organisations hiérarchiques :
- Organigramme d'un service, d'une entreprise, d'une armée (maréchal → généraux → colonels → lieutenants → sergents → troufions)
- Un livre découpé en volumes → tomes → parties → chapitres → sections

14.2 Définition récursive

De manière formelle, un arbre est une structure de données à laquelle on accède par une et une seule donnée, qui donne accès à son information et qui fournit l'accès à plusieurs informations suivantes.

Règle

Définition récursive :
A est un **Arbre** si A est un ensemble tel que :
- il existe un élément particulier appelé **racine**,
- les autres éléments sont partitionnés en ensembles, chaque ensemble étant lui-même un arbre.

14.3 Vocabulaire

- **Nœud** : cellule qui porte une information de l'arbre.
- **Racine** : premier nœud de l'arbre, celui qui donne accès à tous les autres.

— **Sous-arbre** : sous-ensemble d'un arbre qui constitue lui-même un arbre. Tous les nœuds sont racine d'un sous-arbre.
— **Fils** : nœuds accessibles en suivant une et une seule flèche à partir d'un nœud.
— **Père** : nœud qui donne accès à ses fils.
— **Frères** : nœuds ayant le même père.
— **Feuille** : nœud sans fils.

14.4 Caractéristiques

— **Degré d'un nœud** : nombre de fils d'un nœud.
— **Profondeur d'un nœud** : distance d'un nœud à la racine (en nombre de flèches).
— **Hauteur d'un arbre** : maximum des profondeurs des nœuds de l'arbre. Un arbre réduit à un seul nœud est de hauteur 0.
— **Niveau d'un arbre** : ensemble des nœuds se trouvant à une même profondeur.

14.5 Arbres ordonnés

Un arbre est **ordonné** si il existe une relation d'ordre définie sur tous les fils de tous ses nœuds. Les arbres sont très souvent ordonnés.

14.6 Arbres binaires

Les arbres binaires sont des arbres dont **tous les nœuds ont au plus 2 fils**. Ce sont les arbres les plus utilisés.

Règle

Définition récursive d'un arbre binaire :
Un arbre binaire est :
— soit un arbre **vide**,
— soit un triplet constitué d'une information et de 2 arbres binaires : $\langle I, G, D \rangle$
où I représente l'information, G le fils gauche et D le fils droit.

14.7 Implémentation des arbres binaires

Pour représenter des arbres binaires en C++, on utilise **deux classes** : une pour les nœuds et une pour l'arbre global.

```
class noeud {
    int info;
    noeud* fg;     // fils gauche
    noeud* fd;     // fils droit

    friend class arbre;  // arbre peut acceder aux membres prives

public:
    noeud(int I, noeud* G = nullptr, noeud* D = nullptr) {
        info = I;
        fg = G;
```

```
        fd = D;
    }
};
```

Listing 35 – Classe noeud

```
class arbre {
    noeud* racine;

public:
    // Constructeur avec valeur par defaut
    arbre(noeud* N = nullptr) {
        racine = N;
    }
};
```

Listing 36 – Classe arbre

On retrouve bien ici la **récursivité** de la définition : la classe `noeud` est définie par elle-même (ses fils sont aussi des pointeurs sur `noeud`).

Astuce

La déclaration `friend class arbre;` dans la classe `noeud` permet à la classe `arbre` d'accéder aux données privées (`info`, `fg`, `fd`) des nœuds.

14.8 Création d'un arbre

Les constructeurs permettent d'initialiser des arbres directement lors de leur déclaration :

```
arbre A(new noeud(12,
            new noeud(7,
                new noeud(16),
                new noeud(4)
            ),
            new noeud(2)
        )
);
```

Listing 37 – Création d'un arbre binaire

Cet appel crée l'arbre suivant :

```
      12
     /  \
    7    2
   / \
 16   4
```

14.9 Parcours des arbres

On appelle **parcours** d'un arbre le fait d'énumérer en vue d'un traitement quelconque toutes les informations contenues dans l'arbre. La structure récursive des arbres donne une façon simple de réaliser ce parcours.

Règle

Le parcours s'aligne sur la structure récursive de l'arbre :

Définition	Contenu du nœud	Action
Arbre vide	`nullptr`	Rien
Info + 2 arbres	$\langle I, G, D \rangle$	Traiter I, Parcourir G, Parcourir D

```
// Fonction membre privee avec parametre
void arbre::Parcours(noeud* N) {
    if (N == nullptr)
        return;
    cout << N->info << endl;     // Traiter l'information
    Parcours(N->fg);              // Parcourir le fils gauche
    Parcours(N->fd);              // Parcourir le fils droit
}

// Fonction membre publique sans parametre
void arbre::Parcours() {
    Parcours(racine);
}
```

Listing 38 – Fonction de parcours récursive

Important

La fonction de parcours a besoin d'accéder à `racine` (privée dans `arbre`) et aux données des nœuds (`info`, `fg`, `fd`, privées dans `noeud`). D'où la solution :

- Mettre la fonction de parcours comme **fonction membre** de la classe `arbre`
- Déclarer `friend class arbre;` dans la classe `noeud`
- Utiliser **deux fonctions** : une publique sans paramètre (qui appelle avec `racine`), et une privée avec un paramètre `noeud*`

L'appel se fait simplement : `A.Parcours();`

14.10 Exemple de déroulement

Considérons l'arbre suivant :

```
        4
       / \
      9   6
     / \   \
    8   7   5
           / \
          2   1
```

L'appel de `A.Parcours()` provoque `Parcours(racine)` :

1. `N` (racine) n'est pas `nullptr` → affichage de **4**
2. Appel de `Parcours(fg)` → affichage de **9**
3. Appel de `Parcours(fg)` → affichage de **8**
4. `Parcours(nullptr)` → retour, `Parcours(nullptr)` → retour
5. Appel de `Parcours(fd)` → affichage de **7**
6. Retour au nœud 4, appel de `Parcours(fd)` → affichage de **6**
7. Puis **5**, **2**, **1**

Affichage dans l'ordre : **4, 9, 8, 7, 6, 5, 2, 1**

14.11 Les trois types de parcours

Il existe 3 ordres possibles pour les parcours (si on exclut les inversions des fils). Ils ne diffèrent que par l'ordre des lignes dans la fonction :

```
void arbre::ParcoursPrefixe(noeud* N) {
    if (N == nullptr)
        return;
    cout << N->info << endl;        // Traitement
    ParcoursPrefixe(N->fg);         // Fils gauche
    ParcoursPrefixe(N->fd);         // Fils droit
}
```

Listing 39 – Parcours préfixe (racine, puis fils)

```
void arbre::ParcoursInfixe(noeud* N) {
    if (N == nullptr)
        return;
    ParcoursInfixe(N->fg);          // Fils gauche
    cout << N->info << endl;        // Traitement
    ParcoursInfixe(N->fd);          // Fils droit
}
```

Listing 40 – Parcours infixe (fils gauche, racine, fils droit)

```
void arbre::ParcoursPostfixe(noeud* N) {
    if (N == nullptr)
        return;
    ParcoursPostfixe(N->fg);        // Fils gauche
    ParcoursPostfixe(N->fd);        // Fils droit
    cout << N->info << endl;        // Traitement
}
```

Listing 41 – Parcours postfixe (fils, puis racine)

14.12 Répartition des fonctions entre Arbre et Nœud

Une autre organisation possible : une fonction dans `arbre` qui gère le cas de l'arbre vide, et une fonction dans `noeud` qui s'occupe du travail sur un nœud.

```
// Dans la classe arbre
void arbre::Parcours() {
    if (racine == nullptr)
        return;
    racine->Parcours();
}

// Dans la classe noeud
void noeud::Parcours() {
    cout << info << endl;
    if (fg != nullptr)
        fg->Parcours();
    if (fd != nullptr)
        fd->Parcours();
}
```

Listing 42 – Parcours réparti entre arbre et noeud

Astuce

Cette organisation est très élégante mais elle est **forcément récursive**. N'hésitez pas à envisager plusieurs versions de vos fonctions.

14.13 Arbres binaires ordonnés

Un arbre binaire est **ordonné** si pour tous les nœuds de l'arbre :
— tous les nœuds du fils gauche sont **inférieurs** à la racine,
— la racine est **inférieure** à tous les nœuds du fils droit.

```
      6
     / \
    4   9
   / \   \
  1   5   11
   \     /
    2   7
         \
          8
```

Règle

Un **parcours infixe** d'un arbre binaire ordonné énumère les nœuds dans l'ordre croissant.
Par exemple sur l'arbre ci-dessus : 1, 2, 4, 5, 6, 7, 8, 9, 11

14.14 Quelques arbres particuliers

— **Arbre binaire homogène** : tous les nœuds ont 0 ou 2 fils.
— **Arbre binaire dégénéré** : tous les nœuds ont 1 seul fils (sauf le dernier qui n'en a aucun).

— **Arbre en peigne** : arbre dégénéré dont tous les fils sont du même côté. Ce sont en fait des **listes chaînées**.
— **Arbre binaire parfait** : tous les niveaux sont complètement remplis, sauf le dernier qui est totalement vide.
— **Arbre binaire complet** : tous les niveaux sont remplis sauf éventuellement le dernier, et dans ce cas les pointeurs `nullptr` sont à droite.

Règle

Soit un arbre binaire parfait de hauteur k :
— Nombre de feuilles : 2^k
— Nombre total de nœuds : $2^{k+1} - 1$

15 Bonnes pratiques et pièges à éviter

15.1 Checklist des bonnes pratiques

1 Allocation dynamique
— Chaque `new` doit avoir son `delete`
— Chaque `new[]` doit avoir son `delete[]`
— Initialiser les pointeurs à `nullptr`
2 Classes avec ressources dynamiques
— Toujours définir un destructeur
— Toujours définir un constructeur par recopie
— Toujours définir un opérateur d'affectation
— → Règle des trois !
3 Passage de paramètres
— Types simples → par valeur
— Objets en lecture → `const &`
— Objets à modifier → `&`
4 Méthodes const
— Marquer `const` les méthodes qui ne modifient pas l'objet
— Notamment les accesseurs (getters)
5 Encapsulation
— Données membres en `private`
— Méthodes d'interface en `public`
— Utiliser des accesseurs/mutateurs

15.2 Pièges fréquents

Important

Erreurs classiques à éviter :

1. Oublier delete

```
int* p = new int;
*p = 42;
// Oubli de delete p; Fuite memoire !
```

2. Mauvais delete pour un tableau

```
int* tab = new int[100];
delete tab;        // ERREUR ! Il faut delete[]
delete[] tab;      // CORRECT
```

3. Double delete

```
int* p = new int;
delete p;
delete p;          // CRASH !
```

4. Copie superficielle

```
// Sans constructeur par recopie defini
Tableau t1(10);
Tableau t2 = t1;   // t1 et t2 pointent vers le meme tableau
// double delete au destructeur !
```

5. Accès à nullptr

```
int* p = nullptr;
*p = 42;           // CRASH !
```

6. Dépassement de tableau

```
int tab[10];
tab[10] = 5;       // ERREUR : indices 0 a 9 !
```

7. Perdre l'adresse d'une allocation

```
int* p = new int[100];
p = new int[50];   // Perd l'adresse du 1er tableau
// fuite memoire !
```

15.3 Conseils de débogage

Astuce

Pour déboguer les problèmes mémoire :
- Utiliser `valgrind` sous Linux
- Compiler avec `-g` pour avoir les infos de debug
- Ajouter des `cout` dans constructeurs/destructeurs
- Vérifier systématiquement : `if (p != nullptr)`
- Mettre `nullptr` après chaque `delete`

16 Exercices d'entraînement

16.1 Exercice 1 : Classe Rectangle

Énoncé : Créer une classe `Rectangle` avec :
- Attributs privés : `longueur`, `largeur`
- Constructeur avec paramètres
- Méthodes : `aire()`, `perimetre()`, `afficher()`

Solution :

```cpp
#ifndef RECTANGLE_H
#define RECTANGLE_H

class Rectangle {
private:
    double longueur;
    double largeur;

public:
    Rectangle(double l = 1.0, double L = 1.0);
    double aire() const;
    double perimetre() const;
    void afficher() const;
};

#endif
```

Listing 43 – Rectangle.h

```cpp
#include "Rectangle.h"
#include <iostream>
using namespace std;

Rectangle::Rectangle(double l, double L) {
    longueur = l;
    largeur = L;
}

double Rectangle::aire() const {
    return longueur * largeur;
```

```
12 }
13
14 double Rectangle::perimetre() const {
15     return 2 * (longueur + largeur);
16 }
17
18 void Rectangle::afficher() const {
19     cout << "Rectangle " << longueur << "x" << largeur << endl;
20     cout << "Aire : " << aire() << endl;
21     cout << "Perimetre : " << perimetre() << endl;
22 }
```

Listing 44 – Rectangle.cpp

16.2 Exercice 2 : Classe Vecteur dynamique

Énoncé : Créer une classe `Vecteur` qui gère un tableau dynamique d'entiers avec :

- Constructeur avec taille
- Destructeur
- Constructeur par recopie
- Opérateur d'affectation
- Méthode `get(int i)` et `set(int i, int valeur)`

Solution :

```
1 #ifndef VECTEUR_H
2 #define VECTEUR_H
3
4 class Vecteur {
5 private:
6     int* tab;
7     int taille;
8
9 public:
10     // Constructeur
11     Vecteur(int n);
12
13     // Destructeur
14     ~Vecteur();
15
16     // Constructeur par recopie
17     Vecteur(const Vecteur& autre);
18
19     // Operateur d'affectation
20     Vecteur& operator=(const Vecteur& autre);
21
22     // Accesseurs
23     int get(int i) const;
24     void set(int i, int valeur);
25     int getTaille() const;
26
27     // Affichage
```

```
    void afficher() const;
};

#endif
```

Listing 45 – Vecteur.h

```
#include "Vecteur.h"
#include <iostream>
using namespace std;

Vecteur::Vecteur(int n) {
    taille = n;
    tab = new int[n];

    // Initialisation a 0
    for (int i = 0; i < n; i++) {
        tab[i] = 0;
    }
}

Vecteur::~Vecteur() {
    delete[] tab;
}

Vecteur::Vecteur(const Vecteur& autre) {
    taille = autre.taille;
    tab = new int[taille];

    for (int i = 0; i < taille; i++) {
        tab[i] = autre.tab[i];
    }
}

Vecteur& Vecteur::operator=(const Vecteur& autre) {
    if (this == &autre)
        return *this;

    delete[] tab;

    taille = autre.taille;
    tab = new int[taille];

    for (int i = 0; i < taille; i++) {
        tab[i] = autre.tab[i];
    }

    return *this;
}

int Vecteur::get(int i) const {
    if (i < 0 || i >= taille) {
```

```
        cout << "Erreur : indice hors limites" << endl;
        return 0;
    }
    return tab[i];
}

void Vecteur::set(int i, int valeur) {
    if (i < 0 || i >= taille) {
        cout << "Erreur : indice hors limites" << endl;
        return;
    }
    tab[i] = valeur;
}

int Vecteur::getTaille() const {
    return taille;
}

void Vecteur::afficher() const {
    cout << "[";
    for (int i = 0; i < taille; i++) {
        cout << tab[i];
        if (i < taille - 1)
            cout << ", ";
    }
    cout << "]" << endl;
}
```

Listing 46 – Vecteur.cpp

16.3 Exercice 3 : Pile (Stack) avec liste chaînée

Énoncé : Implémenter une pile (LIFO) avec :

- `empiler(int x)` : ajoute x au sommet
- `depiler()` : retire et retourne l'élément du sommet
- `estVide()` : teste si la pile est vide
- `sommet()` : retourne l'élément du sommet sans le retirer

Solution :

```
class Maillon {
    int valeur;
    Maillon* suivant;
    friend class Pile;
public:
    Maillon(int v, Maillon* s = nullptr) : valeur(v), suivant(s
    ) {}
};

class Pile {
private:
    Maillon* sommetPile;
```

```cpp

public:
    Pile() : sommetPile(nullptr) {}

    ~Pile() {
        while (!estVide()) {
            depiler();
        }
    }

    void empiler(int x) {
        sommetPile = new Maillon(x, sommetPile);
    }

    int depiler() {
        if (estVide()) {
            cout << "Erreur : pile vide" << endl;
            return 0;
        }

        int valeur = sommetPile->valeur;
        Maillon* temp = sommetPile;
        sommetPile = sommetPile->suivant;
        delete temp;

        return valeur;
    }

    bool estVide() const {
        return sommetPile == nullptr;
    }

    int sommet() const {
        if (estVide()) {
            cout << "Erreur : pile vide" << endl;
            return 0;
        }
        return sommetPile->valeur;
    }

    void afficher() const {
        cout << "Pile (sommet -> bas) : ";
        Maillon* p = sommetPile;
        while (p != nullptr) {
            cout << p->valeur << " ";
            p = p->suivant;
        }
        cout << endl;
    }
};
```

Listing 47 – Pile.h et Pile.cpp

17 Aide-mémoire

17.1 Tableau récapitulatif des opérateurs

Table 4 – Opérateurs importants en C++

Opérateur	Nom	Utilisation
`&`	Adresse	Donne l'adresse d'une variable
`*`	Déréférencement	Accède à la valeur pointée
`->`	Flèche	Accède à un membre via un pointeur
`.`	Point	Accède à un membre d'un objet
`::`	Résolution de portée	Définit une fonction hors classe
`new`	Allocation	Alloue de la mémoire dynamique
`delete`	Libération	Libère de la mémoire dynamique

17.2 Commandes de compilation

```
# Compilation simple
g++ programme.cpp -o programme

# Compilation avec debug
g++ -g programme.cpp -o programme

# Compilation avec warnings
g++ -Wall -Wextra programme.cpp -o programme

# Compilation de plusieurs fichiers
g++ main.cpp Point.cpp Rectangle.cpp -o programme

# Execution
./programme

# Detection de fuites memoire (Linux)
valgrind ./programme
```

Listing 48 – Compilation et exécution

17.3 Modèle de fichiers .h et .cpp

```
#ifndef NOMCLASSE_H
#define NOMCLASSE_H

```

```
class NomClasse {
private:
    // Attributs prives

public:
    // Constructeurs
    NomClasse();
    NomClasse(/* parametres */);

    // Destructeur
    ~NomClasse();

    // Constructeur par recopie
    NomClasse(const NomClasse& autre);

    // Operateur d'affectation
    NomClasse& operator=(const NomClasse& autre);

    // Methodes publiques
};

#endif
```

Listing 49 – Modèle de fichier .h

```
#include "NomClasse.h"

NomClasse::NomClasse() {
    // Initialisation
}

NomClasse::~NomClasse() {
    // Liberation des ressources
}

// Autres implementations...
```

Listing 50 – Modèle de fichier .cpp

18 Conclusion

Ce document couvre les concepts essentiels du C++ pour le semestre 2 de licence informatique :

- Gestion de la mémoire (pile vs tas)
- Pointeurs et références
- Programmation orientée objet
- Constructeurs et destructeurs
- Allocation dynamique
- Listes chaînées
- Bonnes pratiques

Règle

Les trois règles d'or :

1. **Règle des trois** : Si vous définissez un destructeur, définissez aussi le constructeur par recopie et l'opérateur d'affectation
2. **RAII** (Resource Acquisition Is Initialization) : Acquérir les ressources dans le constructeur, les libérer dans le destructeur
3. **const-correctness** : Utiliser `const` partout où c'est pertinent

Bon courage pour tes révisions !